FindYourBug: An Ensemble-Based Static Bug Localization

Software Requirement Specification

SE - 801 - Software Project Lab III

Submitted by:
Sampad Sikder - 1219

Supervised By
Mridha Md. Nafis Fuad
Lecturer, IIT-DU

Supervisor's Signature:_____



Institute of Information Technology
University of Dhaka

Letter of Transmittal

01 December 2024
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical report on “FindYourBug”.

Sir,
With due respect, I am pleased to submit the technical report of Software Project Lab 3 on “FindYourBug”. I have tried my best to deliver a good understanding about the technical details of the project. However, it might lack perfection. So, I, therefore, hope that you would be kind enough to accept my report and oblige thereby.

Sincerely yours,
Sampad
Roll: BSSE 1219
Institute of Information Technology
University of Dhaka

Acknowledgment

I am grateful to the Institute of Information Technology for giving me such a tremendous opportunity to work on “FindYourBug”. I would like to convey my tremendous gratitude to my supervisor, Mridha Md. Nafis Fuad, Lecturer, Institute of Information Technology, University of Dhaka, for providing me guidelines about how I can prepare this analysis document. I would also like to thank Moumita Asad, Lecturer, Institute of Information Technology, University of Dhaka for managing and overseeing the progress of the project. Lastly, I would like to thank my classmates. They have always been helpful and provided valuable insights from time to time.

Abstract

This document contains the software requirements and specification, architectural design and test plan of “FindYourBug” a static ensemble based fault localizer. This tool will help software developers localize bugs in their source code repository based on the provided bug report. With this, developers can greatly ease their work of locating faults based on a raised issue. The ensemble based approach combines a multitude of techniques to generate the potential list of susceptible files. The approaches are static and don't require the run time trace of the software. The tool takes the source code repository and bug report as input, and using an ensemble based approach outputs the ranked list of susceptible files. The list will be shown on github.

Table of Content

1. Introduction **1**
1.1 Motivation 1
2. Inception **2**
2.1 Proposed System 2
2.2 Methodology 2
2.2.1 Input and Output 2
2.2.2 Preprocessing 3
2.2.3 Fault Localization Techniques 3
2.2.4 Ensemble Model 3
2.2.5 Ranking and Output 4
2.3 Identifying Stakeholders 4
3. Quality Function Deployment **4**
3.1 Normal Requirements 4
3.2 Expected Requirements 5
3.3 Exciting Requirements 5
4. Usage Scenario of FindYourBug **5**
5. Scenario Based Modeling **7**
5.1 Use Case Diagram 7
Level 0: 7
Level 1: 8
Level 1.1: Commit Manager 8
Level 1.2: Code Corpus Creator 9
Level 1.3: Bug Report Manager 10
6. Data Based Modeling **11**
7. Class Based Modeling **11**
7.1 Analysis Classes 11
7.2 Class Cards 11
7.3 Class Responsibility and Collaboration (CRC) Diagram 15
8. Architectural Design **16**
8.1 Architectural Context Diagram 16
8.2 Defining Archetypes 16
8.2.1 Layered Architecture 17
8.2.2 Pipe and Filter Architecture 17
8.2.3 Model-View-Controller (MVC) 17
8.3 Top level components 18
8.4 Instantiation of the Architecture 18
8.4.1 Component: DocumentProcessor 18
8.4.2 Component: SimilarityRanker 19

8.4.3 Component: RepositoryManager 20
8.5 Elaborating the Deployment Diagram 20
9. Preliminary Test Plan 21
9.1 High Level Description of Testing Goals 21
9.2 Test Cases 22
10. Conclusion 27
References 28

List of Figures

Figure 1: A Bug Report 3
Figure 2: Proposed System Flow 4
Figure 3: Level 0 Use Case 7
Figure 4: Level 1 Use Case 8
Figure 5: Level 1.1 Use Case 9
Figure 6: Level 1.2 Use Case 9
Figure 7: Level 1.3 Use Case 10
Figure 8: CRC Diagram of FindYourBug 15
Figure 9: Architectural Context Diagram 16
Figure 10: Top Level Components 18
Figure 11: Layered Architecture of Document Processor 19
Figure 12: Pipe and Filter of Similarity Ranker 19
Figure 13: MVC of Repository Manager 20
Figure 14: Deployment Diagram of FindYourBug 21

1. Introduction

Fault localization is the process of identifying the location(s) in a software system where bugs or faults are likely to exist. It is an essential part of the debugging process, aimed at narrowing down the parts of the codebase that need inspection, making debugging more efficient and reducing the time to resolve issues [1]. Maintaining large-scale software projects requires developers to regularly review bug and error reports submitted by users. With software often comprising thousands of lines of code and hundreds of files, pinpointing the root cause of a bug can be a daunting task. Bug localization techniques can assist developers by identifying areas to investigate. These techniques analyze the source code repository and generate a ranked list of potential files where bugs may reside, significantly easing software maintenance and saving developers valuable time. This greatly eases software maintenance, and saves developers a lot of time.

In broad terms, fault localization can be of two types: Static bug localization and dynamic bug localization. Static bug localization doesn't need the program to be run and relies on information retrieval techniques to generate the ranked list. Dynamic approaches require the program trace and hence require multiple runtime traces of the program [2].

Several static techniques exist to localize bugs, for example: [3]

1. BugLocator
2. AmaLgam
3. BLIA
4. BLUiR
5. BRTracer
6. Locus

Each of these techniques has varying degrees of accuracy, along with distinct advantages and disadvantages in localizing bugs. An ensemble approach that combines these techniques can greatly enhance fault localization. The objective of this project is to develop a GitHub plugin/extension that utilizes this ensemble approach to generate a ranked list of buggy files based on GitHub issues.

1.1 Motivation

Deployed software often receives various issue and bug reports from users. It's the responsibility of software maintenance engineers to fix these issues to ensure the software functions properly. However, identifying the specific files responsible for the bugs can be tedious and time-consuming. In large-scale software systems with numerous files, developers could greatly benefit from tools that guide them to the areas of interest. This is where bug localization

comes in, providing developers with a "map" of potential files to investigate, helping them efficiently narrow down the source of the issue [4].

2. Inception

2.1 Proposed System

The system will be a **GitHub plugin/extension** that employs an ensemble bug localization technique to generate a ranked list of potential buggy files. In GitHub repositories, the **Issues** section is used to report bugs encountered in the software. By providing developers with a ranked list of suspicious files for each issue, they can gain a significant head start in troubleshooting, saving valuable time in the bug-fixing process.

The plugin will take two inputs: the source code repository and the description of the GitHub issue. The output will be a ranked list of files most likely responsible for the reported bugs.

The scope of the project is to

- Take a bug report and code repository as input
- Find the ranked list of suspicious files where bug may occur
- Generate the ranked list based on an ensemble approach of multiple techniques
- Develop a plugin for github
- Generate a ranked list of suspected files based on raised github issues
- Use information retrieval techniques to generate the ranked list
- The technique will not make any mutations in code.

2.2 Methodology

Developers face challenges in locating buggy code files corresponding to bug reports. This process is time-consuming and tedious when dealing with extensive repositories or numerous bug reports. FindYourBug aims to automate bug localization by producing a ranked list of suspicious files using ensemble-based techniques.

2.2.1 Input and Output

Input: Bug reports containing fields such as summary, description, and metadata (e.g., submission date, reporter ID).
Output: A ranked list of source files that are likely to contain bugs, sorted by their suspiciousness scores.

2.2.2 Preprocessing

Corpus Creation: The source code and bug reports are prepared for textual similarity analysis. The preprocessing technique involves performing stop word removal, stemming, tokenization and indexing.

Query construction: Bug report is parsed to construct the query. Mainly, the summary and description field is used.

Bug ID	76138
Open date	2004-10-12 21:53:00
Summary	**Ant editor** not following tab/space setting on shift right
Description	This is from 3.1 M2. I have **Ant**->**Editor**->Display tab width set to 2, insert spaces for tab when typing" checked. I also have **Ant**->**Editor**->Formatter->Tab size set to 2, and "Use tab character instead of spaces _unchecked_. Now when I open a build.xml and try to do some indentation, everything works fine according to the above settings, except when I highlight a block and press tab to indent it. It's the tab character instead of 2 spaces that's inserted in this case.
Fixed Files	org.eclipse.ant.internal.ui.editor.**AntEditor**.java org.eclipse.ant.internal.ui.editor.**AntEditorSourceViewerConfiguration**.java

Figure 1: A Bug Report

2.2.3 Fault Localization Techniques

Various fault localization techniques are used to generate a ranked list of files. The techniques use various Information Retrieval (IR) models to calculate the similarity between score between corpus and query. Each technique then adjusts the score based on weights obtained from version history components.

1. Construct TF-IDF of the code and query.
2. Calculate similarity score using IR techniques.
3. Adjust the score based on history components.

2.2.4 Ensemble Model

The ensemble approach combines the output of the different fault localization techniques.

2.2.5 Ranking and Output

Sort the aggregated scores to produce the final ranked list of files. Highlight the top-k files for further inspection by developers. The generated list will be shown on the plugin.

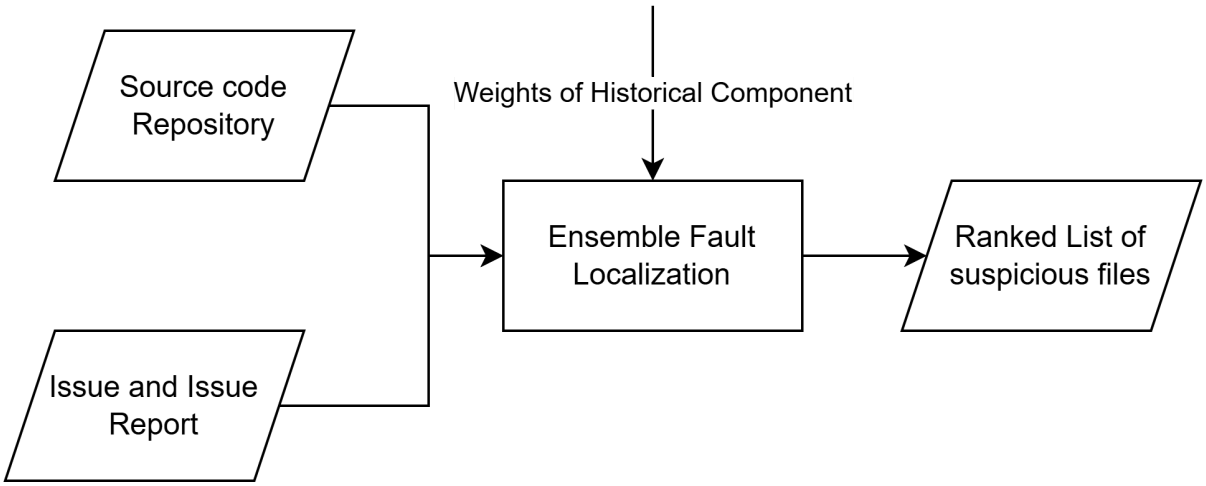


Figure 2: Proposed System Flow

2.3 Identifying Stakeholders

Direct stakeholders:

1. Issue Creator (Bug Reporter)
2. Software Developers/Maintenance Engineers

Indirect stakeholders:

1. Github users
2. Git

3. Quality Function Deployment

3.1 Normal Requirements

These are the general requirements that are stated by the customer during meetings as his goals and objectives of the software. Normal requirements completeness leads to customer satisfaction. For example, requested types of specific system functions. Here are normal requirements:

1. The tool will generate a ranked list of suspicious files from bug reports.
2. The tool will generate a corpus based on the existing source code repository.
3. The tool will parse bug reports to generate bug query.

4. The tool will use information retrieval techniques to generate similarity scores between code repository and bug report.
5. Bug report will be given as input and it will be used as a query to search for suspicious files.
6. The tool will look through all files and no files will be ignored in the search.
7. An ensemble based approach on existing fault localization techniques will be used to generate the ranked list.
8. The tool will be a plugin for Github.

3.2 Expected Requirements

These requirements are so obvious that the customer need not explicitly state them. Their absence can create significant dissatisfaction. The expected requirements of our system are given below:

1. The tool has to show output in the required format necessary.
2. The tool must not make any unnecessary changes to source code.
3. The tool has to be platform independent. (Supported on both PC and mobile platforms)
4. The UI has to be navigable and easy to understand.
5. The tool will not handle any user data.
6. The tool will use a deterministic algorithm, and not involve any learning step.
7. The tool has to be interoperable across different browsers and platforms.

3.3 Exciting Requirements

1. The tool will work on any Github repositories irrespective of language and environment.
2. The suspected files can be accessed on github through the tool.

4. Usage Scenario of FindYourBug

Developers who maintain open source projects need to be wary of issues that are created to report bugs of the project. FindYourBug when configured with a project can easily help developers track down the reported bug.

Issue Creator

After finding a potential bug, an user will go to the repository and create a new issue. For each issue there will be an Issue ID, summary of the issue and description. The issue creator will need to specify for which branch and for which commit the issue has occurred.

Process Initiation
The plugin's ensemble bug localization technique begins its analysis, using multiple static bug localization models (such as BugLocator, AmaLgam, BluIR, etc.). These models examine the codebase structure, bug report keywords, previous bug patterns, and file relevance to locate the probable bug origins.

Document Parser
Bug reports (from issues) are parsed in order to obtain bug id, summary, description, open date branch name and commit id. The process also takes previous bug reports from which the fixed files and commit id are taken. This is used to form the bug history report.

Commit Switcher
To process the codebase, the tool at first checks out the branch and the commit specified. The tool will then use that particular codebase to generate a code corpus.

Text preprocessing
In the text preprocessing step, the corpus is generated from the code, and the query is generated from the bug report. In code, Abstract Syntax Tree is used to parse the code and then class name, method name and statements are specified for each file. The preprocessor performs stop word removal and each word is stemmed and tokenized. For camel case method and class names, each part is separated into individual tokens. Each file is indexed for the query, and the tokens are placed under indices.
The summary and description field of bug report is parsed out, and they are preprocessed to obtain tokens. These tokens are treated as the query.

Similarity score calculation
Using Information Retrieval techniques, the similarity score is determined between the query and the corpus. Various techniques like rVSM, VSM, LDA are used to calculate the similarity score.
Files are then ranked based on their indices using the similarity score.

Generate Ranked List
After analysis, FindYourBug generates a ranked list of files that are likely to contain the bug. The ranked list prioritizes files that have characteristics similar to past issues or that the models suggest as hotspots for the reported problem.

Focused Investigation and Issue Resolution
The developer can navigate to the issue and will find the ranked list of files using the plugin. The developer can now easily navigate to the potentially buggy part of code.

5. Scenario Based Modeling

5.1 Use Case Diagram

A Use Case describes the system behavior under various conditions as the system responds to a request from one of its stakeholders. In fact, a use case diagram is a kind of visualization of the system where an end-user has an idea of a specific feature. It simply describes a story using corresponding actors who perform important roles in the story and makes the story understandable for the users.
The first step in writing a Use Case is to define that set of “actors” that will be involved in the story. Actors are the different people or systems that use the system or product within the context of the function and behavior that is to be described. Actors represent the roles that people play as system operators. They procedure some information or consume some information. Every user has one or more goals when using the system.

Primary Actor: Primary actors interact directly to achieve the required system function and derive the intended benefit from the system. They work directly with the software. They produce some information and consume some information too.

Secondary Actor: Secondary actors support the system so that primary actors can do their work. They either produce or consume information. Here is given the use case diagram to observe the non-technical view of the system.

Level 0:

Name : FindYourBug
Primary Actor: Issue Creator, Developer
Secondary Actor: Github Repository
Goal in Context: The diagram shows the entire FindYourBug tool with the actors.

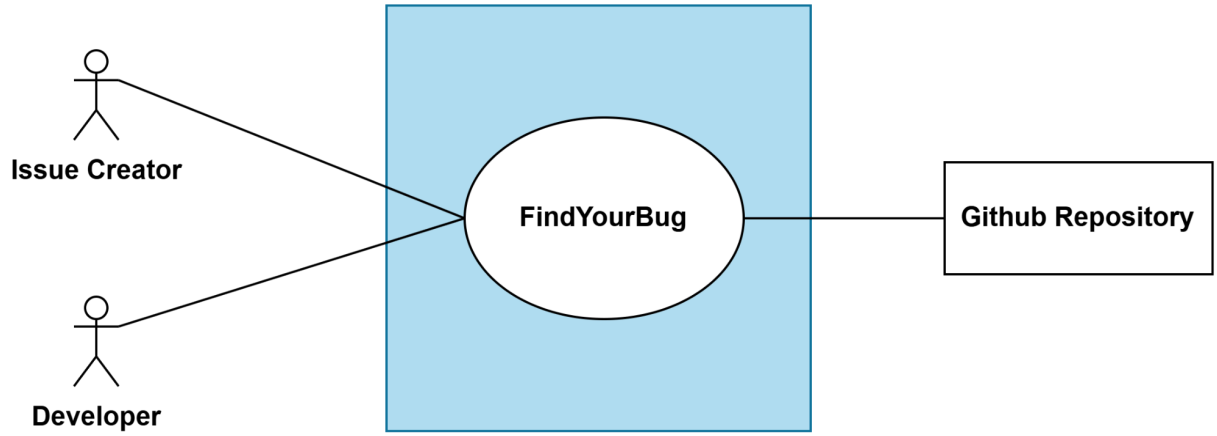


Figure 3: Level 0 Use Case

Level 1:

Name : FindYourBug
Primary Actor: Issue Creator, Developer
Secondary Actor: Github Repository
Goal in Context: The diagram shows all the modules of FindYourBug tool with the actors.

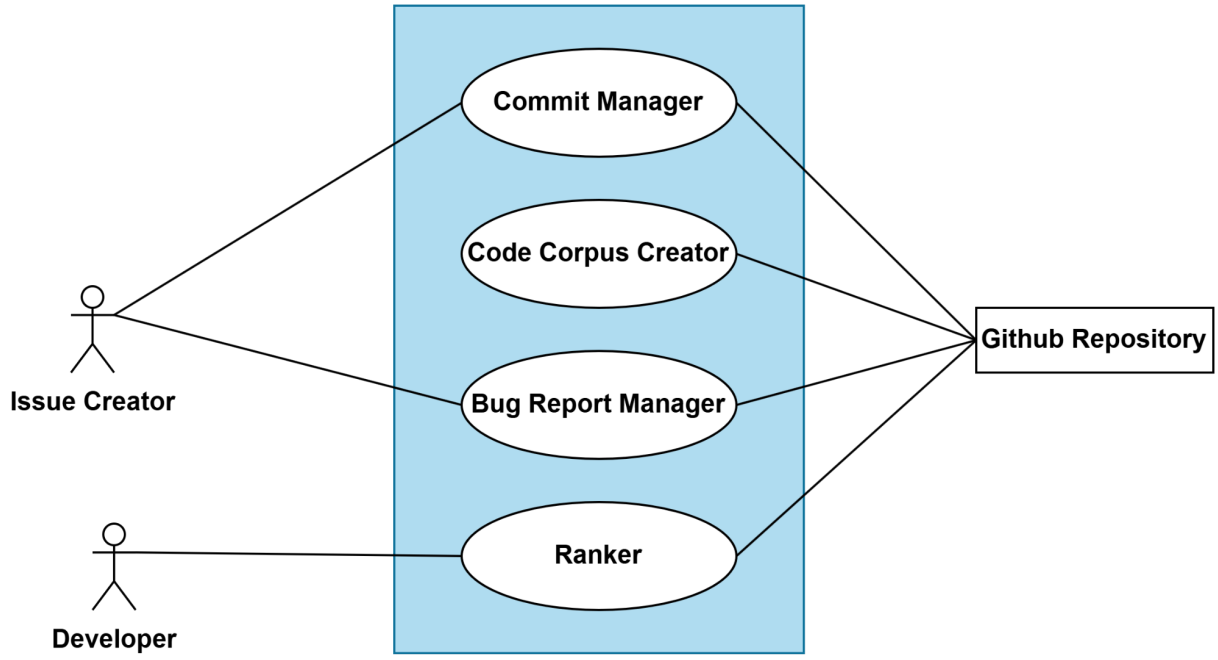


Figure 4: Level 1 Use Case

Commit Manager: Manage the repository commits and branch. Switch to the appropriate branch and commit when necessary.
Code Corpus Creator: Preprocess the code and generate the corpus.
Bug report manager: Parse the bug report and generate the query after preprocessing
Ranker: Calculate the similarity score using Information Retrieval(IR) techniques and rank the files.

Level 1.1: Commit Manager

Primary Actor: Issue Creator
Secondary Actor: Github Repository
Goal in Context: The diagram shows all the modules of the Commit Manager module with the actors.

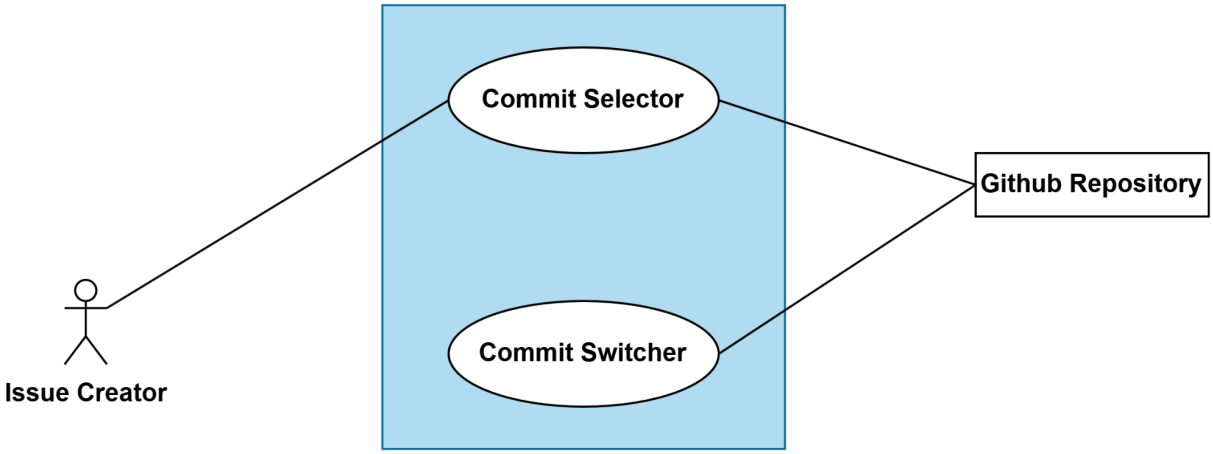


Figure 5: Level 1.1 Use Case

Commit Selector: Issue creator selects the appropriate branch and commit
Commit Switcher: Switcher performs git checkout on the branch and the commit.

Level 1.2: Code Corpus Creator

Primary Actor: There is no primary actor
Secondary Actor: Github Repository
Goal in Context: The diagram shows all the modules of Code Corpus Creator module with the actors.

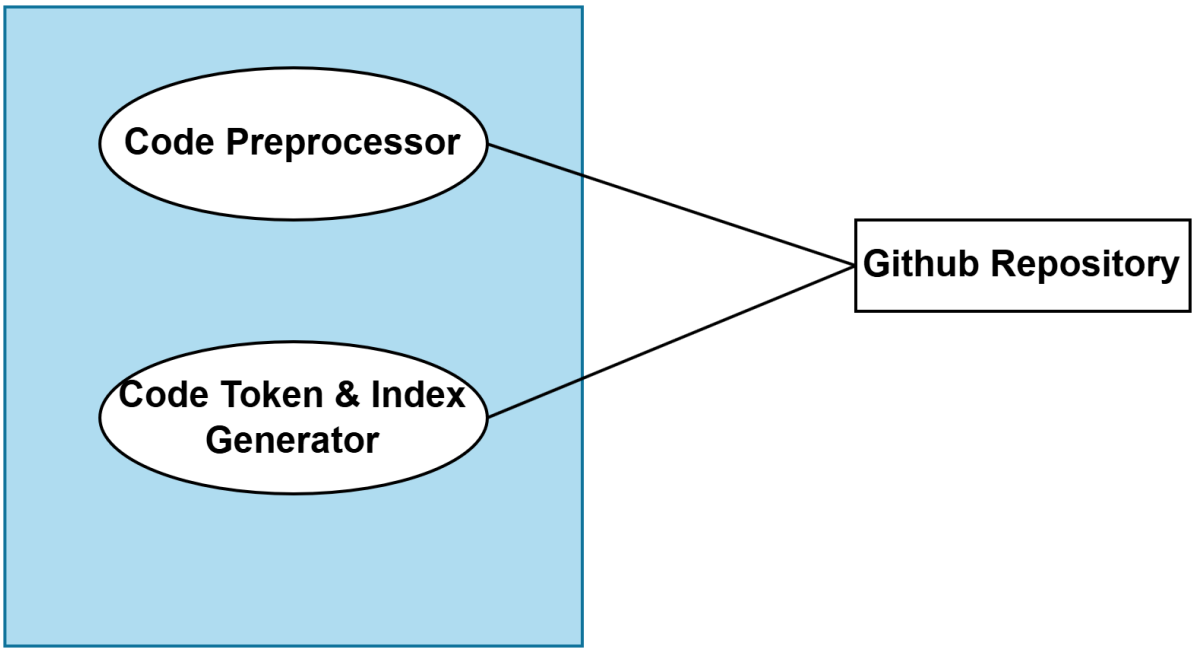


Figure 6: Level 1.2 Use Case

Code Corpus Creator: Perform stop word removal, stemming and tokenizing on the source code files.
Code token & index generators: Make TF-IDF vector of all tokens and generate index for the tokens.

Level 1.3: Bug Report Manager

Primary Actor: Issue Creator
Secondary Actor: There is no secondary actor
Goal in Context: The diagram shows all the modules of the Bug Report Manager module with the actors.

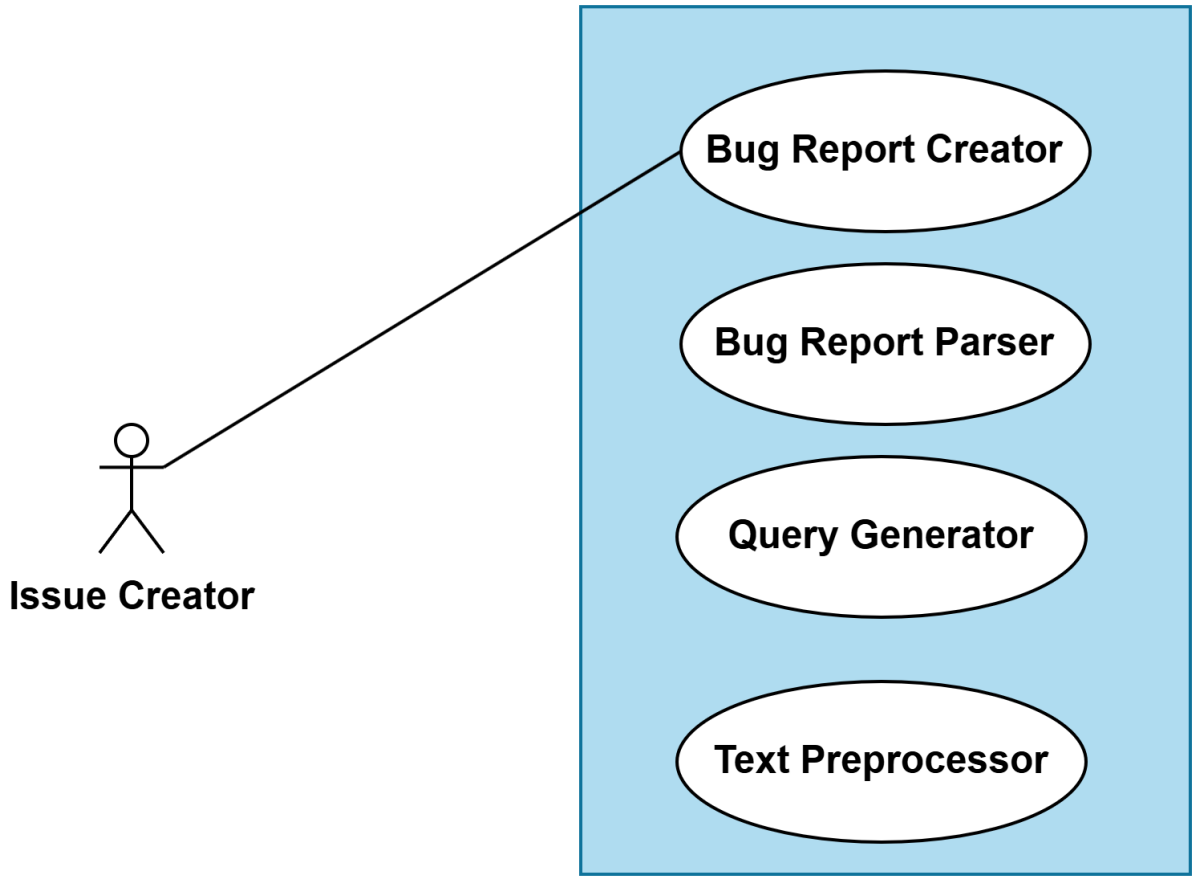


Figure 7: Level 1.3 Use Case

Bug Report Creator: Issue creator creates the bug report with name, id, summary and description.
Bug Report Parser: Parse the report to obtain the different fields necessary for analysis.
Query Generator: Form the query with the analysis fields. These are tokenized versions of the texts.
Text Preprocessor: Before generating the query, preprocess the texts to generate tokens.

6. Data Based Modeling

No data based modeling is needed for this system as the system doesn't use any database or data storage.

7. Class Based Modeling

This chapter describes the class based modeling of the FindYourBug system.

7.1 Analysis Classes

After identifying nouns from the scenario, I have filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes-

1. DocumentParser
2. TextPreprocessor
3. IssueHandler
4. CommitManager
5. QueryGenerator
6. CorpusGenerator
7. SimilarityScoreCalculator
8. Ranker

7.2 Class Cards

Class cards for this tool are documented below-

DocumentParser	
Attributes	**Methods**
+ issueID - bugReport - bugHistory	+ parseCurrentReport(issue) + fetchBugHistory()
Responsibilities	**Collaborators**

- Extract bug details from bug issue (e.g., ID, summary, description, open date, commit ID). - Parse previous bug reports for bug history (e.g., fixed files, commit IDs).	- IssueHandler - TextPreprocessor

TextPreprocessor	
Attributes	**Methods**
- codeCorpus - parsedDocument - queryTokens	+ preprocessCode() + preprocessBugReport() + normalizeText() + stemText(sentence) + tokenizeWords(wordVector)
Responsibilities	**Collaborators**
- Preprocess code and bug reports into tokens. - Perform stop-word removal, stemming, and tokenization.	- QueryGenerator - CorpusGenerator - DocumentParser

IssueHandler	
Attributes	**Methods**
- issueID - branch - commit - summary - description	+ createIssue() + resolveIssue() + showPotentialList()
Responsibilities	**Collaborators**
- Manage the lifecycle of an issue (creation, tracking, resolution). - Validate branch and commit details provided by the user.	- DocumentParser - CommitManager

CommitManager	
Attributes	**Methods**
- currentBranch - currentCommit	+ checkoutBranch() + switchtoCommit()
Responsibilities	**Collaborators**
- Handle branch and commit switching for analysis. - Ensure the correct version of the codebase is analyzed.	- IssueHandler - CorpusGenerator

CorpusGenerator	
Attributes	**Methods**
- codeCorpus - indexedCorpus - query	+ extractASTDetails() + generateCorpus() + indexCorpus()
Responsibilities	**Collaborators**
- Generate the code corpus and query for bug localization analysis. - Parse Abstract Syntax Tree (AST) for method, class, and statement-level tokens.	- TextPreprocessor - CommitManager - SimilarityScoreCalculator

QueryGenerator	
Attributes	**Methods**
- tokenizedDocument - query	+ fetchTokens() + generateQuery()
Responsibilities	**Collaborators**
- Generate Query to match with the Corpus.	- TextPreprocessor - SimilarityScoreGenerator

Ranker	
Attributes	**Methods**
- rankedFiles	+ rankFiles() + showRankOnIssue()
Responsibilities	**Collaborators**
- Rank files based on similarity scores. - Prioritize files with characteristics similar to previous bug patterns.	- SimilarityScoreCalculator - IssueHandler

SimilarityScoreCalculator	
Attributes	**Methods**
- technique - simarityScores - indexList - bugHistoryReport	- applyIRTechniques() - calculateScore() - adjustScoreWithHistory()
Responsibilities	**Collaborators**
- Calculate similarity scores between query and corpus files. - Apply multiple Information Retrieval (IR) techniques (e.g., rVSM, VSM, LDA).	- CorpusGenerator - QueryGenerator - Ranker

7.3 Class Responsibility and Collaboration (CRC) Diagram

The Class Responsibility and Collaboration diagram (CRC) for FindYourBug system is as follows:

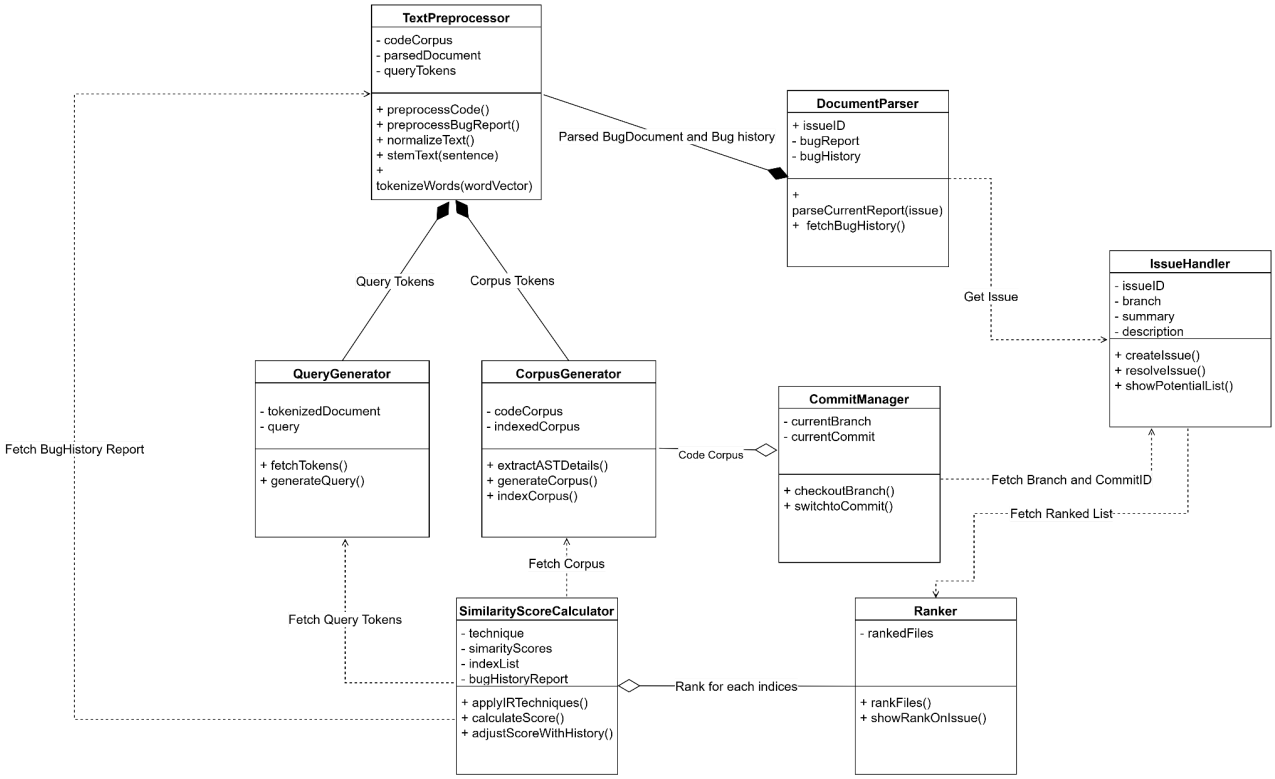


Figure 8: CRC Diagram of FindYourBug

8. Architectural Design

This chapter describes the architectural context diagram, archetypes and top level components for the tool.

8.1 Architectural Context Diagram

The architectural context diagram for the tool is given below-

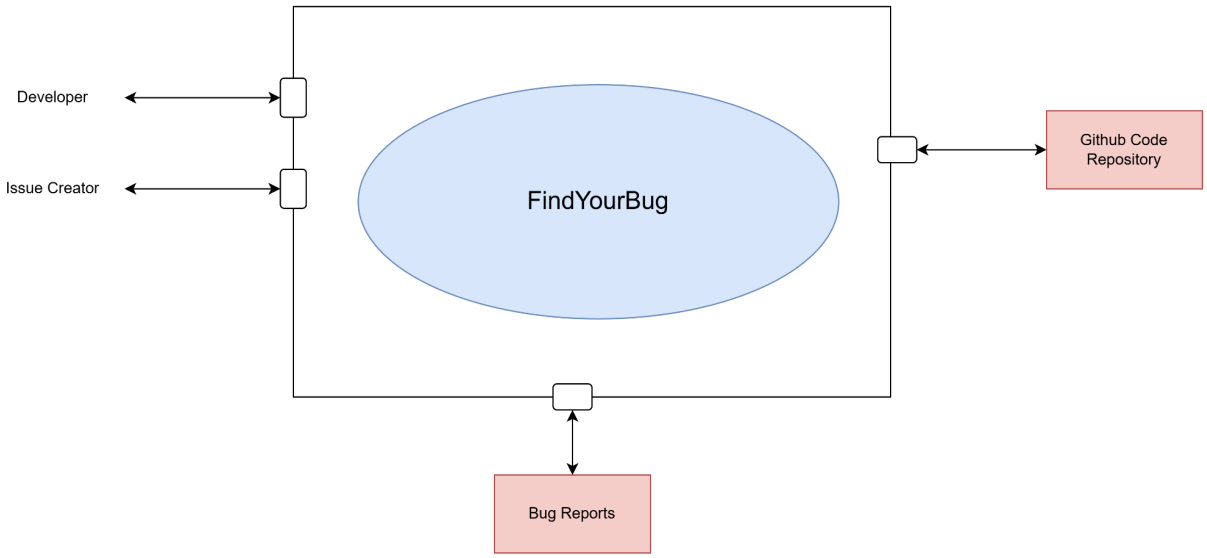


Figure 9: Architectural Context Diagram

Two categories of users are supposed to be the target for the tool. They are software developers and issue creators. Both types of users are the producers and also consumers of the information through the tool. The bug reports obtained from the issue and bug history are the subordinates. The system consumes data from them. The Github repository is the peer level system with the code base being consumed from it and the ranked list of files being delivered to Github.

8.2 Defining Archetypes

The FindYourBug system uses the following archetypes:

1. Layered Architecture
2. Pipe and filter
3. Model-View-Controller (MVC)

8.2.1 Layered Architecture

Layered architecture is a software design pattern that organizes a system's components into distinct layers, with each layer serving a specific set of functionalities and having well-defined interfaces for communication.
It is commonly used in various applications, from web development to enterprise software, due to its clear separation of concerns and ease of maintenance.
Presentation Layer: This layer gets and presents data from and to the system from the user.
Business Logic Layer: Contains the main business logic of our system.
Utility Layer: It contains miscellaneous functions that are used by the business logic layer. It includes library functions, frameworks and well-established algorithms.
Persistence Layer: It contains interfaces to document access objects.
Data Layer: To store the data in storage.

8.2.2 Pipe and Filter Architecture

The Pipe and Filter architecture is a software design pattern that organizes a system as a series of processing elements (filters) connected by communication channels (pipes). It is particularly useful for designing applications that involve processing streams of data in a linear and sequential manner. Each filter performs a specific task on the incoming data and passes the results to the next filter through the pipes, creating a data flow pipeline.

1. **Filter:** All processes that contain a significant general algorithm that can be reused in other tasks. The filters can be re-ordered to solve different problems.
2. **Pipe:** Specific tasks that are required to communicate between the filters.

8.2.3 Model-View-Controller (MVC)

The Model-View-Controller (MVC) framework is an architectural/design pattern that separates an application into three main logical components Model, View, and Controller. Each architectural component is built to handle specific development aspects of an application. It isolates the business logic and presentation layer from each other. It was traditionally used for desktop graphical user interfaces (GUIs).
Model: The Model component corresponds to all the data-related logic that the user works with. This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data.
View: The View component is used for all the UI logic of the application. It generates a user interface for the user. Views are created by the data which is collected by the model component but these data aren’t taken directly but through the controller.
Controller: The Model component corresponds to all the data-related logic that the user works with. This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data.

8.3 Top level components

The overall architecture with top level components are illustrated below. Note that the top level components show a very high level view of our system. Each component has independent responsibilities.

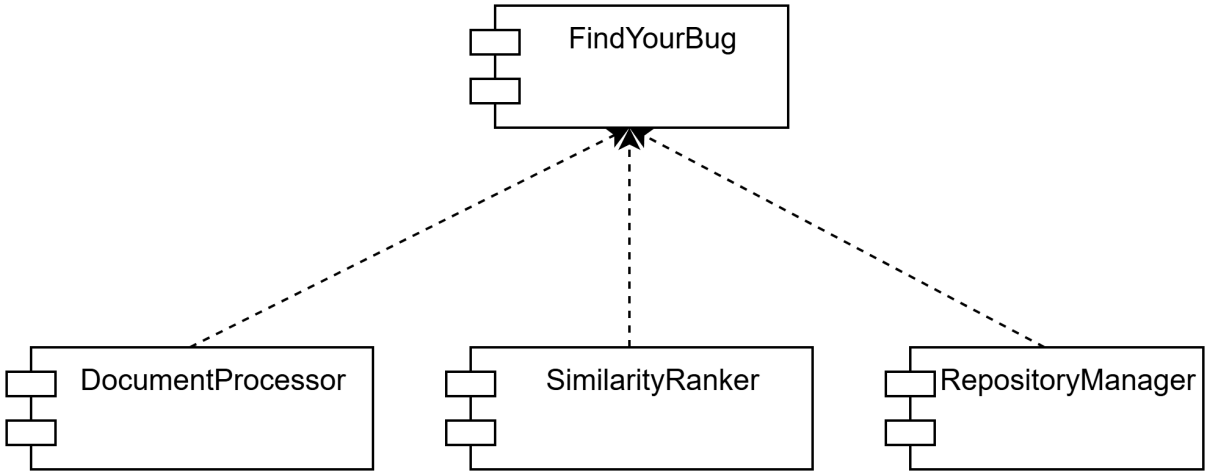


Figure 10: Top level components

8.4 Instantiation of the Architecture

The components identified in Section - 8.3 are broad. So they are refined and instantiated into smaller components. We use different architectural patterns which better suite each component

8.4.1 Component: DocumentProcessor

Pattern: Layered Architecture

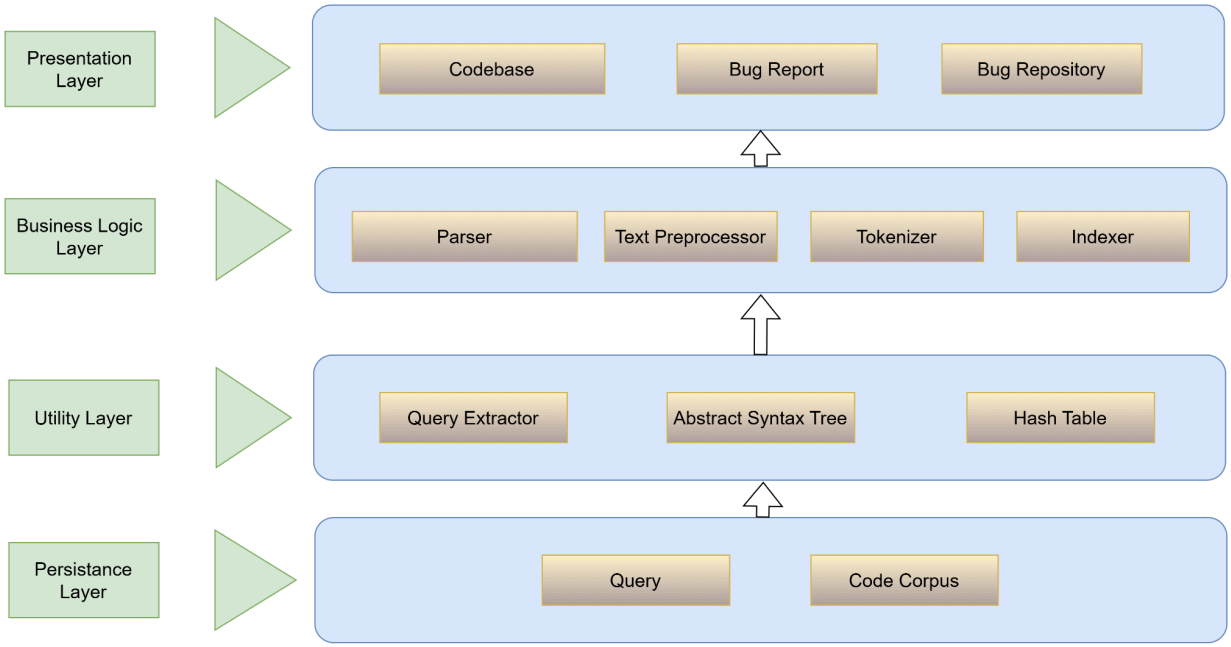


Figure 11: Layered Architecture of Document Processor

The Document Processor component processes the documents (code and bug) to generate query and code corpus. Layered architecture is used for this component as it layers the concerns into different layers, and highlights the required business logic and algorithms needed to carry out the processing.

8.4.2 Component: SimilarityRanker

Pattern: Pipe and Filter

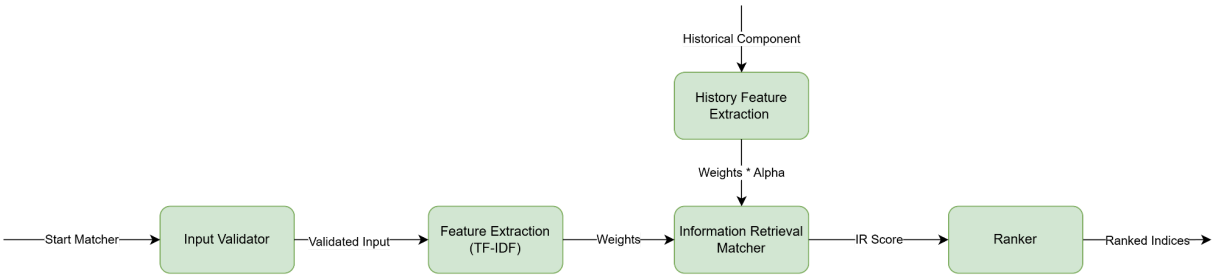


Figure 12: Pipe and Filter of Similarity Ranker

In the SimilarityRanker component, the code corpus and query are sent through a set of pipes and filters and their corresponding ranked indices are obtained at the end. Here the pipes are reusable and scalable.

8.4.3 **Component:** RepositoryManager

Pattern: Model-View-Controller (MVC)

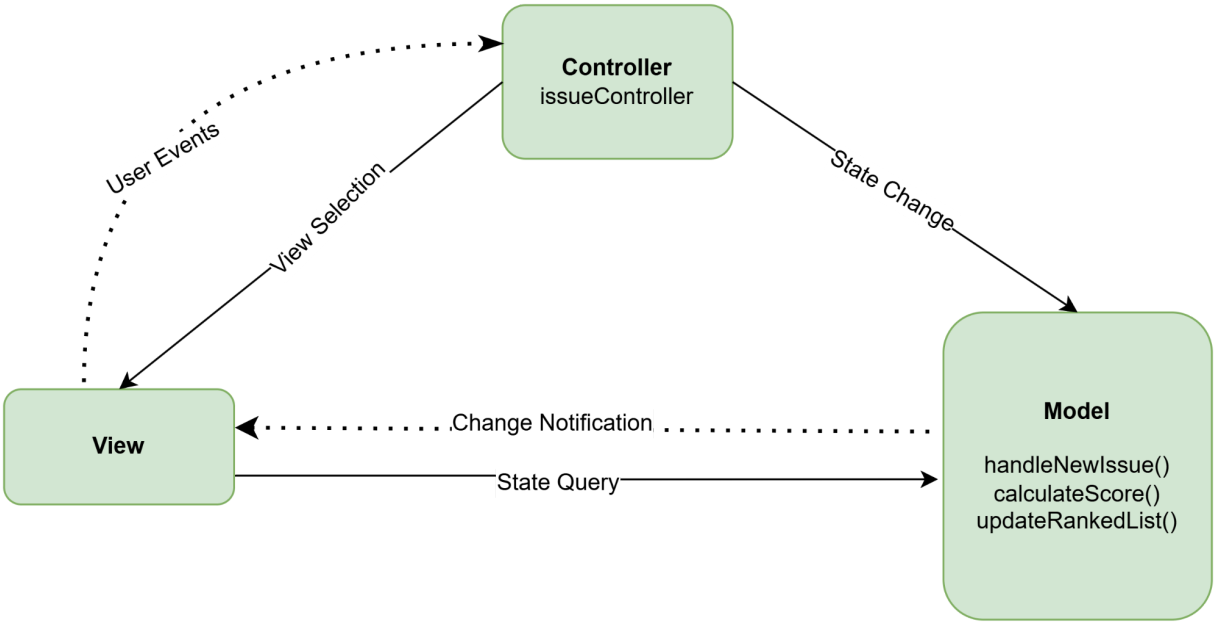


Figure 13: MVC of Repository Manager

The Github Repository manager is best suited for MVC patterns. Here, the issues are placed and the ranked list is shown using view. The model handles the new issue and updates the view with the new ranked list. The controller facilitates communication between model and view.

8.5 Elaborating the Deployment Diagram

The components SimilarityRanker and DocumentProcessor are deployed in two separate servers. The SimilarityRanker node uses multiple techniques to rank files, and so is planned as a server with automated scaling when load increases and faster processing is needed. The DocumentProcessor node receives issues and the code repository from the Github plugin. The plugin is deployed using Jenkins on the Github server.

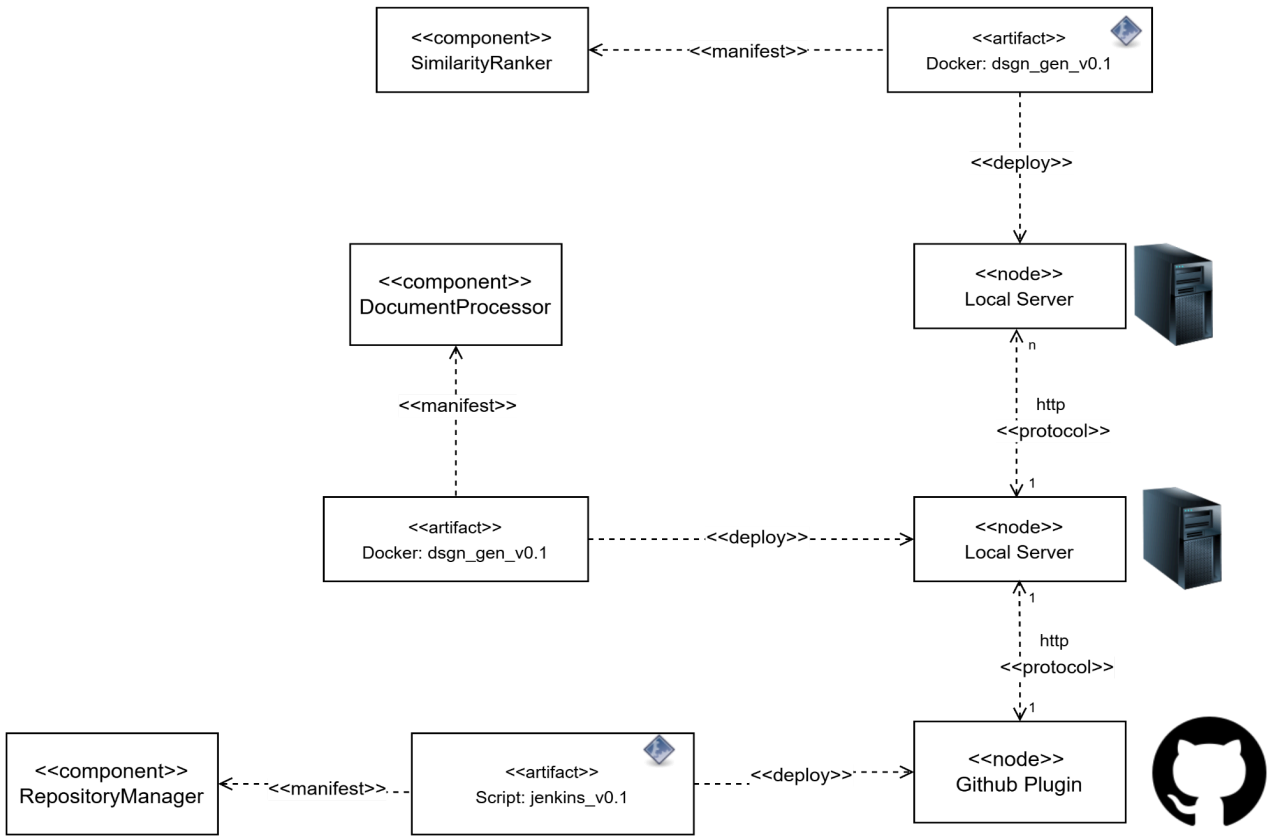


Figure 14: Deployment diagram of FindYourBug

9. Preliminary Test Plan

In this chapter, a high level description of testing goals and summary of features to be tested are presented.

9.1 High Level Description of Testing Goals

Testing goals for this tool is represented below from a high level-

- To demonstrate that the tool meets its requirements
- To ensure that the input codebase is a proper Github repository
- To ensure input is parsed properly
- To ensure that it produces appropriate code corpus and code hunks
- To ensure REST API calls are handled properly
- To ensure that the system properly gives output according to the required format
- To ensure that the system doesn't cause any memory or storage overflow
- To ensure that the tool runs smoothly
- To ensure that the system outputs the correct ranked list of files.

9.2 Test Cases

Test cases for the testing of this tool are documented below-

Test ID: T1

Test Case: Test if source code file is a github repository

Input Test Data: Source code repository

Steps To Be Executed:

1. Run the tool
2. Check if git checkout branch works

Expected Result: Git checkout should run on the project

Target Item: Commit Switcher

Pass/Fail: Pending

Test ID: T2

Test Case: Test if the bug report has all the necessary fields

Input Test Data: Issued bug report

Steps To Be Executed:

1. Run the tool
2. Parse bug report and find if bug id, description, summary, open date and fixed files exist

Expected Result: Parsed bug report should have all the necessary fields

Target Item: Document Parser

Pass/Fail: Pending

Test ID: T3

Test Case: Make sure that output is properly converted to JSON format

Input Test Data: Ranked list of files

Steps To Be Executed:

1. Run the tool
2. Print the ranked list and convert it to JSON format

Expected Result: JSON formatted output should have all necessary field for the github plugin

Target Item: Issue Handler

Pass/Fail: Pending

Test ID: T4

Test Case: Make sure the code corpus is being generated properly

Input Test Data: Code repository

Steps To Be Executed:

1. Run the tool
2. Check the code corpus, code index and code hunk text files

Expected Result: Generated temporary text files should have valid items

Target Item: Corpus Generator

Pass/Fail: Pending

Test ID: T5

Test Case: Commit ID of bug report is not found in the code repository

Input Test Data: Code repository and commit ID

Steps To Be Executed:

1. Run the tool
2. Check if code repository can switch to the commit

Expected Result: Code repository can switch to the commit

Target Item: Commit Switcher

Pass/Fail: Pending

Test ID: T6

Test Case: Commit ID of bug report is not found in the code repository

Input Test Data: Code repository and commit ID

Steps To Be Executed:

1. Run the tool
2. Check if code repository can switch to the commit

Expected Result: Code repository can switch to the commit

Target Item: Commit Switcher

Pass/Fail: Pending

Test ID: T7

Test Case: Tool is triggered when an issue is created

Input Test Data: Create a new issue

Steps To Be Executed:

1. Run the tool
2. Create an issue

Expected Result: Tool starts to run when an issue is created

Target Item: Issue Handler

Pass/Fail: Pending

Test ID: T8

Test Case: Bug History reports are configured properly

Input Test Data: Bug history reports

Steps To Be Executed:

1. Run the tool
2. Parse the bug history report to obtain commit id, fixed files, summary and description

Expected Result: Bug history have all the necessary fields

Target Item: Text Preprocessor

Pass/Fail: Pending

Test ID: T9

Test Case: Result ensemble outputs proper score

Input Test Data: Ranked list of different techniques (if multiple techniques are ran)

Steps To Be Executed:

1. Run the tool
2. Collect the result of all different techniques

Expected Result: If multiple techniques are run, there should be a single ranked list comprising of the result of all techniques

Target Item: Ranker

Pass/Fail: Pending

Test ID: T10

Test Case: System takes configurations properly

Input Test Data: Configurations of the system - input code repository, bug report, alpha value and technique to be execute

Steps To Be Executed:

1. Run the tool
2. Configure the techniques

Expected Result: All configurations were provided

Target Item: Similarity Score Calculator

Pass/Fail: Pending

Test ID: T11

Test Case: RestAPI calls are being handled properly

Input Test Data: API call

Steps To Be Executed:

1. Run the tool
2. Send or receive API call with appropriate headers

Expected Result: Status code received 200 OK

Target Item: Issue Handler

Pass/Fail: Pending

10. Conclusion

FindYourBug is an innovative and developer-centric tool designed to revolutionize the process of fault localization. By leveraging an ensemble of advanced techniques, including textual similarity, structural modeling, and version history analysis, it efficiently pinpoints buggy files and provides developers with a ranked list of suspicious code. The tool's interactive dashboard, seamless integration with version control systems, and dynamic bug triaging capabilities streamline debugging workflows and reduce time-to-fix, ultimately enhancing productivity.

References

[1] J. Zhou, H. Zhang and D. Lo, "Where should the bugs be fixed? More accurate information retrieval-based bug localization based on bug reports," 2012 34th International Conference on Software Engineering (ICSE), Zurich, Switzerland, 2012, pp. 14-24, doi: 10.1109/ICSE.2012.6227210

[2] M. Wen, R. Wu and S. -C. Cheung, "Locus: Locating bugs from software changes," 2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE), Singapore, 2016, pp. 262-273.

[3] Shaowei Wang and David Lo. 2016. AmaLgam+: Composing Rich Information Sources for Accurate Bug Localization. J. Softw. Evol. Process 28, 10 (October 2016), 921–942. https://doi.org/10.1002/smr.1801

[4] R. K. Saha, M. Lease, S. Khurshid and D. E. Perry, "Improving bug localization using structured information retrieval," 2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE), Silicon Valley, CA, USA, 2013, pp. 345-355, doi: 10.1109/ASE.2013.6693093.

TRASH

8.5 Mapping Requirements to Software Architecture

The mapping of the derived components with the requirements can be shown below:

Requirements	**Components**
• NR - The tool will generate a ranked list of suspicious files from bug reports. • NR - The tool will use information retrieval techniques to generate similarity scores between code repository and bug report. • NR - An ensemble based approach on existing fault localization techniques will be used to generate the ranked list. • EX - The tool has to show output in the required format necessary. • EX - The tool will use a deterministic algorithm, and not involve any learning step.	InformationRetrieval Matcher, Ranker (Pipe and Filter)
• NR - Bug report will be given as input and it will be used as a query to search for suspicious files.	BugReport (Presentation Layer)
• NR - The tool will look through all files and no files will be ignored in the search. • NR - The tool will parse bug reports to generate bug query. • NR - The source program needs not to be executed to find the suspicious files • NR - The tool will generate a corpus based on the existing source code repository.	Codebase (Presentation Layer), BugReport (Presentation Layer) Parser, Tokenizer, Text Preprocessor, Indexer (Business Logic Layer), Query Extractor, AST, Hash Table (Utility Layer)
• EX - The tool must not make any unnecessary changes to source code.	InputValidator (Pipe and Filter)
• NR - The tool will be a plugin for Github. • EX - The UI has to be navigable and easy to understand. • EX - The tool has to be platform independent. (Supported on both PC and mobile platforms) • EX - The tool will not handle any user data. • EX - The tool has to be interoperable across different browsers and platforms.	RepositoryManager View, RepositoryManager Controller, RepositoryManager Model

• Exc - The tool will work on any Github repositories irrespective of language and environment. • Exc - The suspected files can be accessed on github through the tool.	